

INDEXES

Manuela Vázquez

November 2025

1 Indexes

Indexes in a database are structures used to improve query performance and speed up the data search process. Without indexes, when a query is executed, the system would have to search sequentially through all existing records, which becomes very costly as the volume of data grows.

With the presence of indexes, we can quickly locate the records that are relevant to the queries we want to execute. Depending on the type of query and the characteristics of the data, we will use one type of index or another. This allows filtering, sorting, or joining (JOIN) operations to be executed more efficiently.

Let's apply this indexing theory to our database project for the "Music Streaming Platform".

Let us retrieve our queries.sql file from Deliverable 2, which contains all the queries we have executed so far on our database. Indexes are mainly based on the queries being executed, because the main purpose of an index is to reduce the query search time.

When we index an attribute, what we do is create a structure that allows PostgreSQL to find records much faster without having to read them all. It is important to look at our queries to decide which columns would benefit from an index and which would not. For example, a column that contains a wide variety of values would benefit from a B-tree index, which sorts the data and makes searches, sorts, and filters more efficient.

Therefore, our first step will be to observe the queries and identify which columns appear most frequently or which, due to their nature, would greatly benefit from an index in order to reduce execution time. We should examine which columns appear more often in WHERE clauses, JOIN conditions, GROUP BY operations, ORDER BY statements, etc. Based on this analysis, we will be able to decide which indexes are most effective for our database.

By analyzing our 25 queries, we can clearly see that the columns most frequently used in WHERE, JOIN, equality searches, and similar operations are the following:

Before reviewing the columns used in our queries, it is important to clarify that the primary keys of each of our relations already have a built-in B-Tree index created automatically. Although it may seem at first glance that they require an index to improve efficiency, our PostgreSQL database engine is responsible for ensuring their uniqueness and fast lookup. Therefore, we do not need to consider any primary key when designing our indexes.

Column	Use in Queries	Index
user_nick	query 6 – GROUP BY query 10 – GROUP BY query 16 – LIKE query 20 – GROUP BY query 24 – GROUP BY	B-Tree
email	query 16 – LIKE	B-Tree
country	query 4 – GROUP BY query 8 – GROUP BY AND ORDER BY query 11 – GROUP BY query 24 – GROUP BY	B-Tree
date_birth	query 16 - BETWEEN	No

Table 1: Table Users

Column	Use in Queries	Index
user_id	query 2,4,5,6,9,10,11,13... - JOIN query 9 - GROUP BY	B-Tree
song_id	query 1,5,8,9,24... - JOIN query 9,21 - GROUP BY	B-Tree
user_id, song_id	query 5,6,8,10,11,12,14,21,24...	Composite index
play_date	query 1 - WHERE query 2 - HAVING query 13 - ORDER BY	B-Tree
device_id	query JOIN,ORDER BY	B-Tree

Table 2: Table Play_History

Column	Use in Queries	Index
song_title	query 1,3,5,12,15,21-WHERE, GROUP BY, ORDER BY	B-Tree
album_id	query 19- JOIN	B-Tree

Table 3: Table Song

Column	Use in Queries	Index
artist_name	query 1,3,5,12,15,21-WHERE, GROUP BY, ORDER BY	B-Tree

Table 4: Table Artist

Column	Use in Queries	Index
user_id	query 4,5,21- JOIN	B-Tree

Table 5: Table Playlist

Column	Use in Queries	Index
(song_id, genre_id)	query 3,20,25- JOIN	Composite Index

Table 6: Table Song_Genre

Column	Use in Queries	Index
(song_id, artist_id)	query 9,14,22,25- JOIN	Composite Index
song_id	query 4,5,21- JOIN	B-Tree
genre_id	query 4,5,21- JOIN	B-Tree

Table 7: Table Song_Artist

We could also add an index on `user_id`, since it is a foreign key in `User_Free` and `User_Premium`

We have listed the main columns that appear in the queries, and in the `indexes.sql` file we have the indexes that were created along with the justification for each one. Now we will continue with the analysis of these indexes. As we have already mentioned several times, creating indexes speeds up the queries in our database, but it also has a negative side that we will discuss in this appendix—these are the well-known trade-offs (disadvantages) of creating indexes.

Obviously, the main benefit of creating indexes is that `SELECT` queries become much faster; likewise, `JOIN` operations, sorting, and filtering also become more efficient. Operations involving range conditions or equality comparisons also become significantly lighter.

The main drawbacks arise when inserting new data, because every time a new record is added to the database, the associated index must be updated, which makes the process slower. The same happens with `UPDATE` statements, since each modification requires reinserting the index, and also with `DELETE` operations. Additionally, indexes usually occupy a considerable portion of the database size.

Below we analyze the trade-offs of 10 of the indexes we have created:

1. **INDEX (user_id, play_date)**

The main advantages of creating this composite index are that queries involving users and date ranges become much faster; and many of our queries rely on this type of filtering. It also allows PostgreSQL to perform an `ORDER BY` on `play_date` after filtering by `user_id` without a costly manual sort. In general, it improves queries related to listening time, recent activity, and temporal analysis. The main disadvantages are that—since it is more complex than a simple index—it takes up more space, increases the cost of `INSERT` operations, and if `play_date` does not appear together with `user_id`, the index does not provide any benefit.

2. INDEX song_id-play_history

Several queries in our database calculate total song play counts, so this index speeds up those queries. It is also very useful for ORDER BY, filtering operations, and JOINS that appear across multiple queries. The only drawback is that it takes up space in the database, and if we filter by user or by playlist, the index does not provide any benefit.

3. INDEX device_id-play_history

This index is very useful for queries that filter or group by device. However, if the variety of devices in our database is very small, the index may become counterproductive.

4. SIMPLE INDEX playlist_id (play_history)

This index helps speed up JOINS and improves certain queries related to playlist playback. It can also be useful for statistics such as playlist duration or number of reproductions. However, it is not very useful in most of the other queries.

5. **INDEX: user_id,song_id** For the composite index (user_id, song_id), we can highlight several advantages: both attributes appear together in many queries; it is very useful to determine how many times a user has listened to a song; it is also frequently used in JOINS and GROUP BY operations and complements the other composite index we have created. The main disadvantage is that, being a composite index, it has a higher cost and is only useful for queries that use both attributes simultaneously.

Basically, we have addressed some advantages and disadvantages of certain indexes, but most of them share the same benefits. Indexes slow down data update operations, while they speed up data retrieval operations.

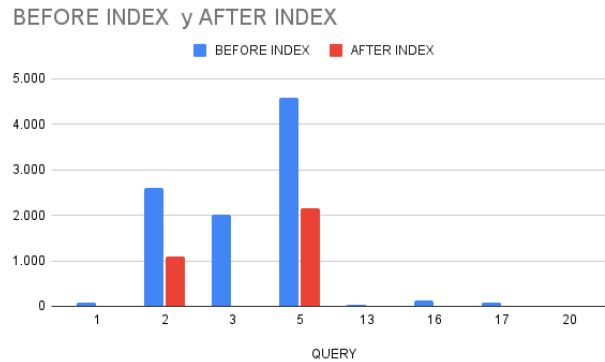
Now we will move on to the index testing phase, focusing on evaluating the real costs of index creation. In this section, we will analyze performance improvements in SELECT queries, as well as the overhead introduced during INSERT and UPDATE operations. We will also assess which type of index is most beneficial for each query (e.g., B-Tree, Hash) and put into practice the trade-offs and index strategy considerations discussed earlier. The benchmark results will be summarized in comparative tables, while the detailed execution plans and in-depth analysis will be documented separately in the indexes file.

Query	Tipo de búsqueda	Índice usado	T. SeqScan (ms)	T. Index (ms)	%	Observaciones
QUERY 1	GROUP BY + ORDER BY + LIMIT	(user_id, play_date) B-Tree	7.013	1.474	79	
QUERY 13	JOIN,WHERE,ORDER BY	(user_id, song_id) B-Tree	2.649	0.397	85	Escanea NULLs
QUERY 17	JOIN/WHERE	user_nick,user_id	7.780	0.155	98	Índice reducido
QUERY 16	ORDER BY, WHERE, BETWEEN	date.birth	12.665	1.647	87	Hash más rápido no ordena

Table 8: Comparativa de tests de índices

As shown in our comparative table, the use of indexes in queries significantly reduces waiting times. Additionally, the indexes.sql file provides examples using HASH indexes and illustrates how time cost increases during INSERT and UPDATE operations due to index maintenance overhead.

We conclude that HASH indexes are the best option for equality lookups, while B-Tree performs better in range queries and ordering. However, indexes add extra write cost due to maintenance, which becomes noticeable in bulk INSERT and UPDATE operations...



To conclude this section, we will clarify two things: -The execution times of each query vary every time we run them, but what really matters is that, as shown in the chart, they are significantly lower after applying the indexes. -The queries used to generate the INDEX performance charts are the ones that were most positively impacted by the indexes.